Calculating Urban Heat Island Effect with Python

By: Alayna Breden
AES 508
Spring 2026
https://github.com/adb0061-lang/AES_508_Final

Project Goals

- Calculate the temperature rise in urban areas of Huntsville from 2019-2024.
- Create a function that will read in the raster data and clip it to Huntsville boundary
- Create a function that will take the clipped raster data of the temperature maps, calculate ndvi maps, and display them side by side

Data Sources

- Using data from USGS Earth Explorer, I downloaded imagery from 2019 to 2024, all from roughly the same date in August
  - https://earthexplorer.usgs.gov/
- To find the Huntsville city boundary, I used Huntsville Maps
  - https://maps.huntsvilleal.gov/datadepot/?path=GIS_data_depot_shapefiles
- I read the shapefile into python using geopandas

```
boundary_path = "HuntsvilleCityLimits/Boundary_City_Huntsville_poly.shp"
LS09_data_folder = "Landsat_Data/"
city = gpd.read_file(boundary_path)
```

Methodology

- Land Surface Temperature maps can be found using band 10 off landsat satellite 8/9
- Original code I used the “*.TIF”
  - The issue with this is that there are a lot of TIF files and I only needed B10.
- The fixed glob pattern used “*B10.TIF”
- This gave me a list of all of the B10 raster filepaths for each year I had

```
b10_tif_files = glob.glob(LS09_data_folder + "*B10.TIF")
b10_tif_files
```

```
[156]:

['Landsat_Data\\LC08_L2SP_020036_20190815_20200827_02_T1_ST_B10.TIF',
 'Landsat_Data\\LC08_L2SP_020036_20230826_20230905_02_T1_ST_B10.TIF',
 'Landsat_Data\\LC08_L2SP_020036_20240828_20240905_02_T1_ST_B10.TIF',
 'Landsat_Data\\LC09_L2SP_020036_20220831_20230331_02_T1_ST_B10.TIF']
```

clip_to_city Function

- This function takes a raster TIF file and clips it to a given boundary shapefile
- Uses `rasterio` to open the TIF
- Reprojects the given boundary to the same coordinate reference system as the raster TIF
- Uses `rasterio.mask` to clip the raster data to the reprojected boundary shapefile geometry
- Returns both the clipped layer and the transform

extract_year and get_path Function

- I needed to calculate NDVI for each year in my dataset using band4 and band5 TIF files
- Instead of hard coding the file names for each year, I wrote a function to use to find the band4 and band5 files for a given year.
- I used `re.search` to pull the year out of the B10 TIF file paths I am using in my main loop.
    - Admittedly, I had to get help writing this regular expression
- Once I had the year able to be extracted, I wrote a function to substitute the ST_B10 part of the TIF path with a given band. This returns a file path string for my band4 and band5 files.

compute_ndvi Function

- The formula for NDVI is (NIR - RED) / (NIR + RED)
- I wrote a function to compute this, however I was running into errors.
  - Some of the numbers in my raster data had zeroes, leading to divide by zero issues
- I used `np.where` to handle the divide by zero issues

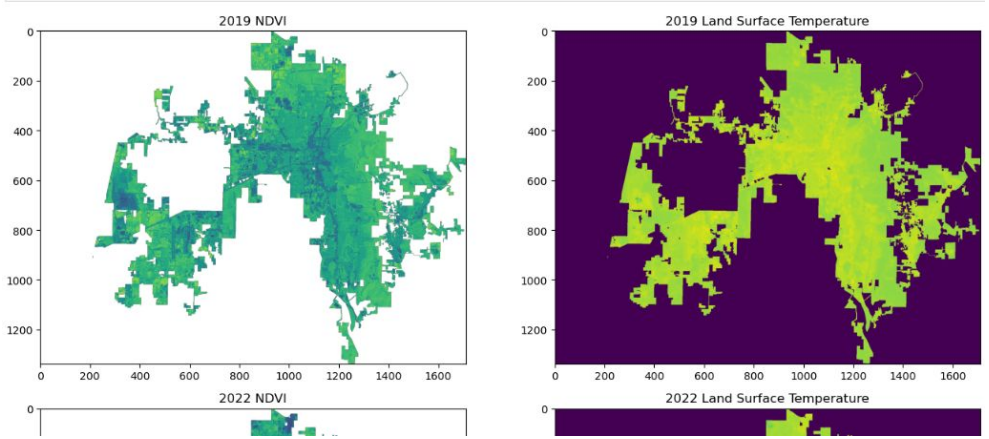
```
def compute_ndvi(b4, b5)
    # to fix divide by 0 errors
    b4 = np.where(b4 == 0, np.nan, b4)
    b5 = np.where(b5 == 0, np.nan, b5)
    denom = (b5 + b4)
    return np.where(denom != 0, (b5 - b4) / denom, np.nan)
```

Getting Each Years Results

- I have four years of data: 2019, 2022, 2023, and 2024
- I loop over the four B10 TIFs
- For each, I used the `get_path` function to get band 4 and band 5 paths
- Next I clipped B10 to the Huntsville boundary shapefile and then did the same for B4 and B5
- Then I used B4 and B5 to calculate the ndvi for each year using `compute_ndvi`
- Then I saved the clipped data to a dictionary keyed off of the year extracted from the B10 TIF file

Plotting Results

- I sorted the data by years using the dictionary previously created
- I made a two column plot with the ndvi on the left and the land surface temperature on the right
- I then looped over the dictionary to organize the maps horizontally by year starting with 2019
- I plotted using `matplotlib`



Analysis

- The results do show an increase in temperature with the more urbanized areas as the years go by
- It was difficult to get the colors to change properly so I left them as the default.
- It is to note that for the 2023 data there was more cloud coverage compared to the other years. This coverage makes the map look cooler than the temperatures really were for that day.
    - I did a mean temperature and 2023 was lower than the other years even though it showed a steady increase.
- Huntsville is growing rapidly and the maps show the increase in urbanization. This increase leads to an increase of heat within the city.

Future Work

- While working on this project, I found that there was a study done from NASA Develop with a very similar premise to mine covering the years 2013-2019.
- If I were to continue this project, I would expand it with additional years data using earlier landsat satellites and compare my results to the NASA Develop project results.
- I would also like to use this for other major cities in the country and compare it to the neighboring rural areas.
- I would like to try to calculate numerically the changes from year to year, rather than relying on visual analysis.

Python

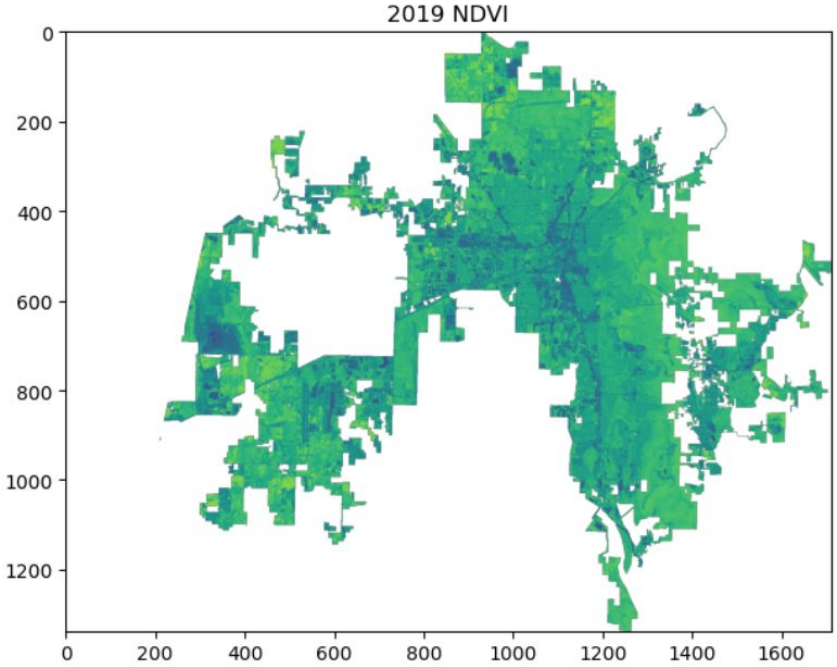
In this project, I used multiple different python tricks that we learned in this class:

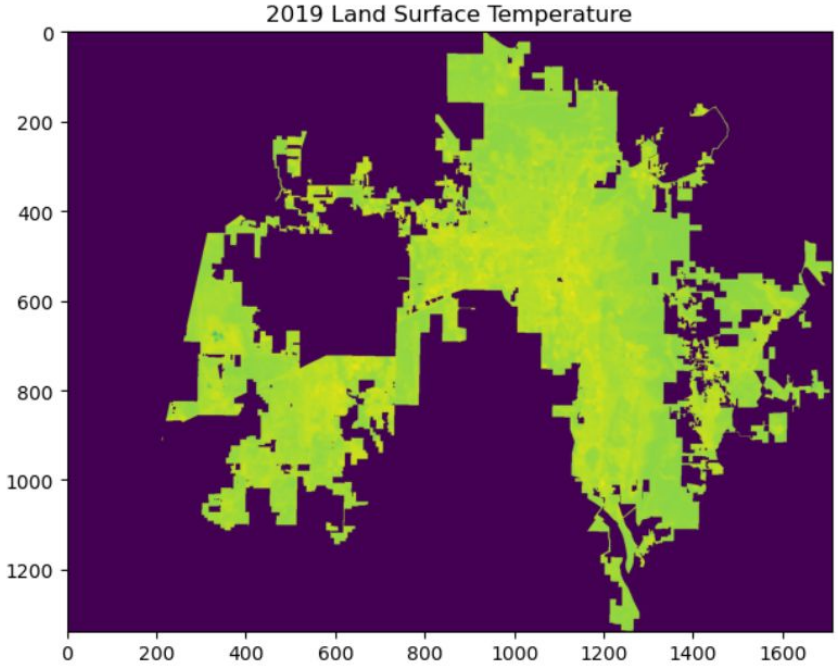
- Functions
- Loops
- Dictionaries
- Raster data processing with `rasterio`
- Masking and Clipping
- Plotting using `matplotlib`

I had to learn some new things specifically for this project:

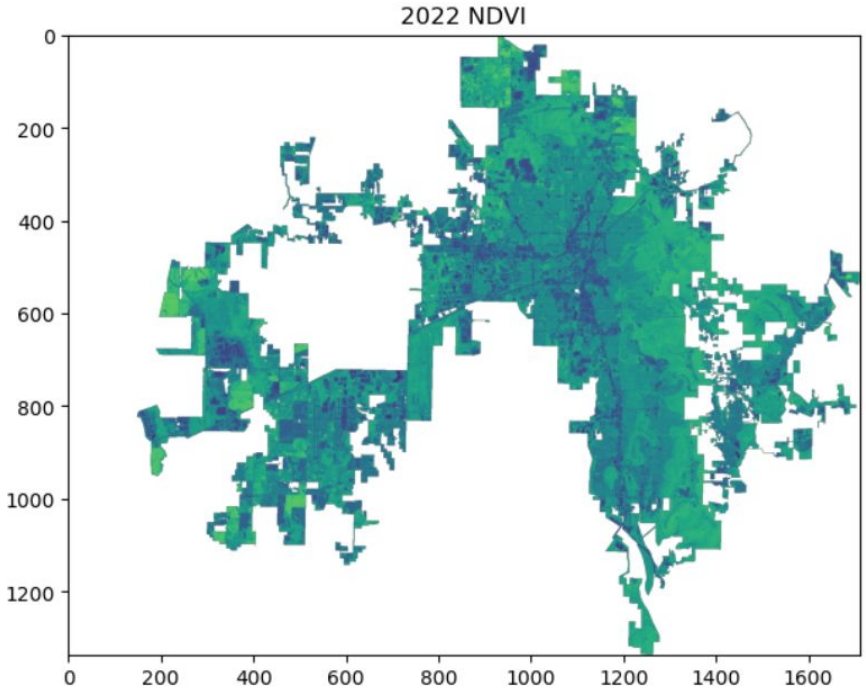
- Regular expressions with `re` and `glob`

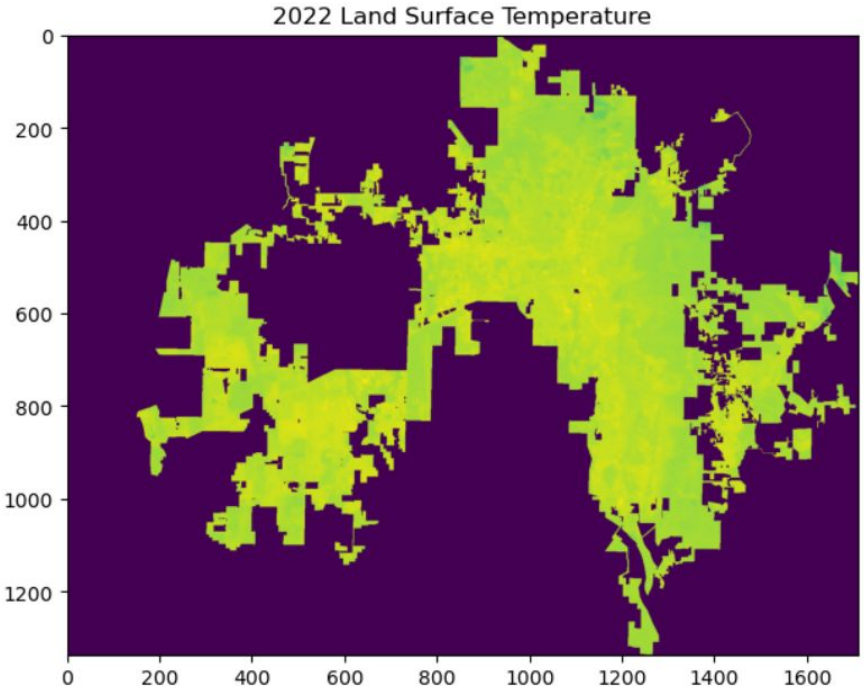
2019 Results



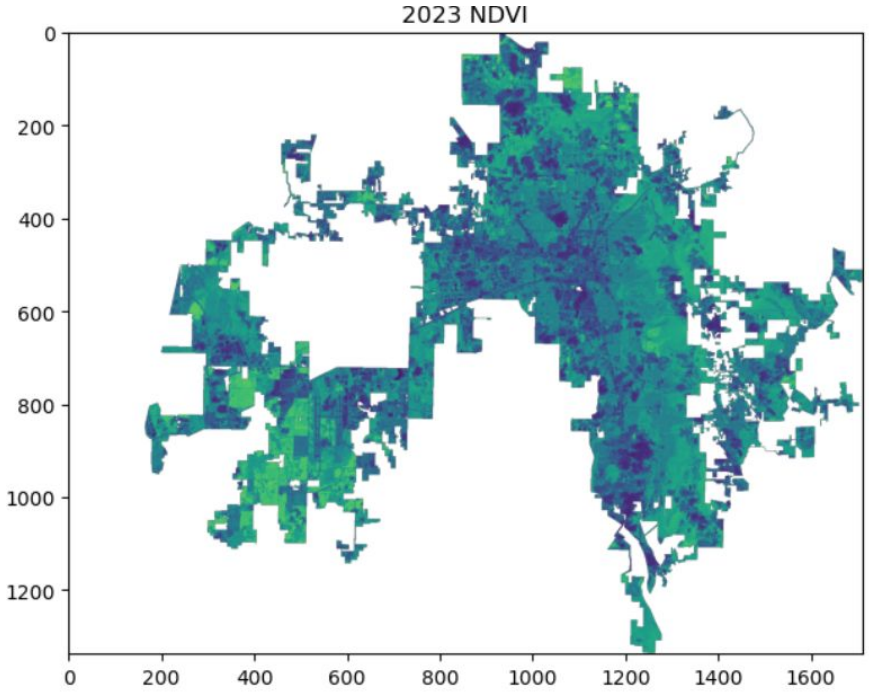


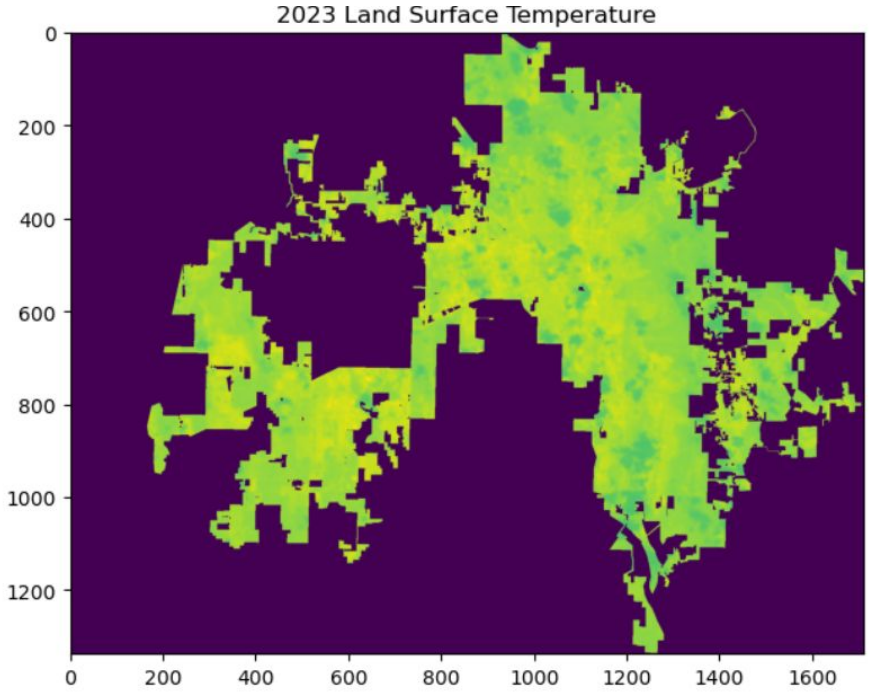
2022 Results





2023 Results





2024 Results